LEADERS UNIVERSITY / LICENCE BI

Du fichier au rapport : je construis ma BI

Atlas Papeterie · Python, ETL et entrepôts de données · 21 h guidées + 7 h de défi

28 h	70 %	0
AU TOTAL	PRATIQUE	PRÉREQUIS IT

ATLAS PAPETERIE

Fichiers de ventes → ETL → Entrepôt → Rapport

02 / CONSTRUIRE SA BI AVEC IA

Youssef Douma · Leaders University

Édition pédagogique · 27 octobre 2026 · données fictives

LE PROJET / DU FICHIER AU RAPPORT

Construire une BI que je peux défendre

Atlas Papeterie est une entreprise fictive avec deux boutiques : Nabeul (NBE) et Tunis (TUN). La responsable prépare une discussion sur les achats : quels produits ont été vendus, dans quelle boutique et pour quel montant ? Les fichiers reçus contiennent aussi des répétitions et des erreurs. Votre travail est de construire un chemin fiable jusqu'à un rapport local.



- Vous écrirez de petits contrats en français et ferez produire du code avec une IA.
- Vous enregistrerez ce code dans projet/, puis exécuterez et contrôlerez chaque étape.
- Votre rapport sera produit depuis les données ; vous saurez retrouver la source d'un chiffre.

Le résultat ne décide pas seul

Les ventes observées ne donnent ni stock disponible, ni coût, ni marge. Une période courte ne prédit pas la demande. Votre conclusion doit rester dans ce que les fichiers permettent de vérifier.

COMPRENDRE AVANT DE CONSTRUIRE

Python, ETL, BI : trois outils pour une question

Un programme est une suite d'instructions enregistrées dans un fichier. Python permet ici de lire du texte, appliquer des règles et interroger une base. Vous n'installez aucune bibliothèque de calcul : la bibliothèque standard fournit csv, decimal, datetime, sqlite3 et pathlib.

Mot	Ce qu'il fait ici
BI	Business Intelligence : transformer des données en information utilisable et explicable.
ETL	Extract : lire ; Transform : contrôler et calculer ; Load : ranger dans la base.
Entrepôt	Organiser des faits de ventes et leurs descriptions pour répondre à des questions.
SQLite / SQL	Un fichier de base local ; SQL est le langage qui permet d'en interroger les tables.

Python a été conçu comme un langage général lisible ; sa documentation décrit les structures et modules employés ici. SQLite fournit un moteur SQL intégré au programme. Nous choisissons ces outils pour garder une installation légère et voir tout le chemin d'une vente.

Repères officiels : https://docs.python.org/3/tutorial/ et https://www.sqlite.org/about.html. Le livre 01 traite l'installation et les exigences de votre poste.

Votre première explication

Dire avec vos mots : le CSV contient des lignes reçues ; la base contient les faits acceptés ; le rapport présente des agrégats contrôlés. Ce sont trois états différents de la même activité.

28 HEURES / UN SEUL PROJET QUI GRANDIT

L’itinéraire : sept missions puis un défi

Mission	Production concrète	Temps
01 · Calculer	Un total et une variation vérifiés	3 h
02 · Lire	16 lignes et un dictionnaire des colonnes	3 h
03 · Nettoyer	12 retenues + 1 doublon + 3 rejets	3 h
04 · Modéliser	Quatre tables vides reliées par des clés	3 h
05 · Charger	12 faits, 32 articles, 99.000 TND	3 h
06 · Ajouter	14 faits et un rejeu stable	3 h
07 · Rapporter	Un HTML issu de SQLite et une consigne de reprise	3 h
CampusBox	Adapter, prouver, expliquer ; équipe de 1 à 3	7 h

Prévision : 504 minutes de théorie et 1 176 minutes de pratique. Chaque mission guidée prévoit 54 minutes de concepts et 126 minutes de manipulation, lecture active, essais, dépannage et explication. Le défi prévoit 126 + 294 minutes. L’installation fait partie de la mission 01.

Une durée de travail, pas un chronomètre

Les durées sont des estimations de conception. Elles n’ont pas été mesurées auprès de débutants. Les pauses s’organisent dans le budget de la séance ; les extensions restent facultatives.

MÉTHODE / PRÉDIRE → CONSTRUIRE → PROUVER

Le geste qui revient à chaque mission

1 Prédire → 2 Demander → 3 Exécuter → 4 Expliquer

- Lire le petit exemple et calculer le résultat attendu avant de consulter une réponse IA.
- Ouvrir le fichier annoncé dans l'éditeur. Une demande produit une fonction ou un petit fichier.
- Lire ce que la réponse importe, lit et écrit. Coller uniquement le code dans ce fichier et enregistrer.
- Revenir au terminal, lancer la commande de la mission, puis comparer la sortie à la prédiction.
- Tester une variation proche et expliquer pourquoi le résultat change. Garder les CSV originaux intacts.

La fenêtre de discussion aide à écrire ; le terminal exécute réellement le programme sur votre poste. Une réponse persuasive, une capture ou un texte « tout fonctionne » ne remplace jamais la sortie locale du vérificateur.

Votre dossier de preuves

Créer projet/journal_apprentissage.md dans l'éditeur : une section par mission avec le contrat, une prédiction, une sortie réelle et une phrase d'explication. Le code et ce journal restent sur votre ordinateur.

AVANT LA MISSION 01 / LIRE LE LIVRE 01

Préparer un espace où je sais agir

Télécharger puis extraire le dossier complet du cours. Ouvrir ce dossier dans l'éditeur, pas seulement un PDF. Il contient compose.yaml, projet/, outils/, solutions/ et les livres dans pdf/. Dans Windows, afficher les extensions : calcul.py ne doit pas devenir calcul.py.txt.

Zone	Rôle
Éditeur	Ouvrir projet/calcul.py, lire et modifier le texte, puis Ctrl+S.
PowerShell Windows	Ouvrir à la racine, là où se trouve compose.yaml ; y saisir les commandes docker compose.
Terminal Ubuntu	Ouvrir au même niveau ; utiliser les mêmes commandes Docker.
Navigateur	Discussion IA et consultation du rapport à http://localhost:8000/rapport_atlas.html.

Terminal · racine du dossier du cours

```
docker compose build
docker compose run --rm atelier python --version
```

Le livre 01 explique Docker Desktop sous Windows, Docker Engine et Compose sous Ubuntu, ainsi que l'installation native de secours. Si Docker est indisponible, ouvrir un terminal dans projet/ et remplacer la commande Docker par py -3 sous Windows ou python3 sous Ubuntu : par exemple py -3 verifier.py 01. Conserver cette même route pour toute la séance.

Tout ne doit pas déjà fonctionner

Avant votre première construction, le vérificateur peut signaler NotImplementedError : cela indique un fichier à compléter. Un message « fichier introuvable » indique plutôt le mauvais dossier ou un nom mal enregistré.

Ne remplacer que la fonction annoncée

Chaque fichier contient les imports, les chemins et un main() déjà fournis. Remplacer seulement def de la mission et son bloc TODO/NotImplementedError. Garder les autres fonctions et le lancement en bas du fichier. Les prompts demandent cette seule définition.

MISSION 01 / UN BESOIN MESURABLE

01 · Le panier qui révèle un programme faux

La responsable annonce deux carnets à 4.500 TND chacun et trois stylos à 1.500 TND chacun. Avant de programmer, identifiez ce qui est une quantité et ce qui est un prix. Le montant d'une ligne est la quantité multipliée par le prix unitaire ; le panier est la somme de ces montants.

Produit	Quantité	Prix / unité	Montant
Carnet	2	4 500 millimes	9 000 millimes
Stylo	3	1 500 millimes	4 500 millimes
Total			13 500 millimes

Une fonction est une petite machine nommée : elle reçoit une entrée et renvoie une sortie. Notre fonction calculer_total reçoit une liste de ventes ; elle doit renvoyer un entier en millimes. Elle ne demande rien au clavier et n'affiche pas le total à la place de le renvoyer.

Prédiction à écrire

Si le nombre de carnets passe de 2 à 3, le total devient 18 000 millimes. La différence est un carnet, soit 4 500 millimes. Ne lancez pas encore le programme.

MISSION 01 / LISTE, DICTIONNAIRE, NOMBRE

Les objets Python du panier

Un dictionnaire relie un nom à une valeur : quantity vaut 2. Une liste regroupe plusieurs dictionnaires, donc plusieurs ventes. Les crochets repèrent la liste, les accolades un dictionnaire et les guillemets une clé de texte. Un entier tel que 4500 n'a pas de guillemets.

Python · petit exemple autonome

```
ventes = [
    {"quantity": 2, "unit_price_millimes": 4500},
    {"quantity": 3, "unit_price_millimes": 1500},
]
print(ventes[0]["quantity"])
print(2 * 4500 + 3 * 1500)
```

Créer docs/exemples/01_panier.py à la racine du cours avec ce code. Avec Docker, lancer la commande ci-dessous. Sortie attendue : 2 puis 13500. Le premier élément de la liste porte l'indice 0 ; cette convention est utilisée souvent en programmation.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python ../docs/exemples/01_panier.py
```

La commande tient sur une seule ligne, sous PowerShell comme sous Ubuntu. En mode natif depuis projet/, utiliser py -3 ../docs/exemples/01_panier.py ou python3 ../docs/exemples/01_panier.py.

Unité exacte

1 TND = 1 000 millimes. Stocker 13500 évite de confondre 13.500 TND et 13 500 TND. L'affichage en TND viendra après le calcul.

MISSION 01 / UNE FONCTION, UNE SORTIE

Mon premier contrat pour l'assistant

Ouvrir le fichier projet/calcul.py dans l'éditeur. Il contient un emplacement à compléter. Dans la discussion web de votre choix, copier la demande ci-dessous ; le livre 03 donne aussi les demandes prêtes à copier. Aucun abonnement ni terminal IA n'est nécessaire pour suivre le cours.

Demande 01 · à copier

Écris seulement la définition de calculer_total(ventes), en Python standard. Je remplacerai uniquement le bloc def/TODO de projet/calcul.py et conserverai main() et le bas du fichier. Fonction calculer_total(ventes) : ventes est une liste de dictionnaires contenant quantity et unit_price_millimes, deux entiers positifs. Renvoyer un entier égal à la somme quantity × unit_price_millimes, sans input(), sans afficher à la place du retour, sans lire ni écrire de fichier. Une liste vide renvoie 0. Exemple : 2 × 4500 et 3 × 1500 donne 13500 ; avec 3 carnets le total est 18000. Conserver exactement le nom et les arguments. Ajouter deux commentaires utiles, sans dépendance externe.

Lire avant de coller

- Repérer def calculer_total(ventes) : c'est l'entrée attendue par les étapes suivantes.
- Repérer la multiplication ; une simple somme des prix est incorrecte.
- Repérer return : c'est la sortie que le vérificateur pourra comparer.
- Coller le code sans les marques de mise en forme ``` ; enregistrer avec Ctrl+S.

Ne demandez pas les missions 02 à 07 dans la même réponse. Votre contrôle du petit contrat est plus important que la taille du programme reçu.

MISSION 01 / PREMIÈRE EXÉCUTION RÉELLE

Faire travailler le code enregistré

Dans PowerShell ou le terminal Ubuntu à la racine du cours, saisir la commande. Le conteneur exécute le fichier enregistré dans projet/. Une modification oubliée dans l'éditeur n'est pas encore un changement du programme exécuté.

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python calcul.py
docker compose run --rm atelier python verifier.py 01
```

Observation	Interprétation / action
Contrôle réussi	Le panier, la liste vide et la variation sont compatibles avec le contrat. Lire aussi le code.
NotImplementedError	Le fichier contient encore le repère à compléter ; coller puis enregistrer.
Résultat 6000	Le code additionne 4500 et 1500 : il ignore les quantités.
SyntaxError / IndentationError	Lire la dernière ligne de l'erreur ; vérifier les guillemets, deux-points et espaces.

Si le résultat est 6000, le programme peut être exécutable et faux. Envoyer à l'IA seulement la fonction, l'entrée, attendu 13500 et obtenu 6000. Demander : « Quelle opération manque ? Corrige cette fonction seulement et conserve son interface. » Enregistrer puis relancer exactement le même contrôle.

Une différence pédagogique

Le candidat fautif à 6000 est un exemple construit pour apprendre à vérifier ; il ne décrit pas une réponse effectivement observée d'un fournisseur d'IA.

MISSION 01 / DU RÉSULTAT À LA COMPRÉHENSION

Changer une chose et expliquer l'écart

Retourner dans le petit exemple docs/exemples/01_panier.py. Remplacer la première quantité par 3 et adapter le calcul de la dernière ligne pour représenter cette même entrée. Écrire 18000 avant de relancer. Ensuite, remettre l'exemple initial ; conserver dans le journal les deux résultats.

Essai	Prédiction en millimes	Pourquoi
2 carnets + 3 stylos	13 500	Deux montants sont additionnés.
3 carnets + 3 stylos	18 000	Un carnet ajoute 4 500.
Aucune vente	0	Aucun montant à additionner.

Question de lecture : quel objet contient les deux ventes ? Quelle clé contient la quantité ? Pourquoi return total permet-il au programme suivant de réutiliser le nombre alors que print(total) écrit seulement à l'écran ?

Si je termine tôt

Créer sur papier une vente à 4 stylos. Prédire 6 000 millimes pour cette seule vente puis tester via une petite copie de l'exemple. Ne modifiez pas le vérificateur pour lui faire accepter votre fonction.

Prévoir 35 minutes d'installation dans la séance, puis un premier cycle de 45 minutes consacré au panier. Le reste du temps sert à lire, varier, corriger et expliquer. Un poste déjà préparé permet de consacrer ce temps à la lecture du contrat.

MISSION 01 / CONTRÔLER ET REPRENDRE

Jalon 01 · une preuve que je peux expliquer

À ce stade, je sais...

- distinguer quantité, prix unitaire et montant ;
- représenter une vente par un dictionnaire et plusieurs ventes par une liste ;
- enregistrer une fonction, lancer un contrôle et refuser un résultat faux même sans erreur Python.

Conserver une preuve courte

Conserver calcul.py, la sortie de verifier.py 01 et deux lignes du journal : 13 500 pour le panier initial, 18 000 pour la variation.

Expliquer avant de continuer

Pourquoi le candidat à 6 000 semble-t-il plausible ? Montrer l'information qu'il a oubliée dans l'entrée.

Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 01 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

MISSION 02 / EXTRAIRE

02 · Recevoir des lignes sans perdre leur sens

Le panier devient un fichier de ventes. Un CSV est un texte tabulaire : une ligne d'en-tête donne les noms des colonnes, puis chaque ligne décrit une observation. Une cellule vide, des espaces ou un code inconnu restent des informations reçues ; la lecture ne décide pas encore de les accepter.

Colonne	Sens
sale_id	Identifiant stable d'une vente élémentaire.
date	Date de cette vente au format AAAA-MM-JJ.
shop_id / product_id	Code de boutique / produit, relié à un catalogue.
quantity	Nombre d'articles du même produit sur cette ligne.
unit_price_tnd	Prix d'un article en TND, écrit avec un point décimal.

Une ligne représente un produit vendu dans une boutique à une date. Elle ne prouve pas un client unique, une commande complète ou un paiement. Ce sens de la ligne s'appelle le grain ; toutes nos sommes doivent le respecter.

Première inspection

Ouvrir projet/data/ dans l'éditeur et repérer le fichier initial ainsi que les catalogues de produits et boutiques. Lire le texte sans le réenregistrer avec un tableur, qui pourrait transformer les dates et séparateurs.

Les fichiers sont projet/data/ventes_initiales.csv, ventes_ajout.csv, produits.csv et boutiques.csv. La période initiale est du 5 au 7 janvier 2026 ; après ajout, elle se termine le 8 janvier 2026.

MISSION 02 / LE CSV RESTE DU TEXTE

Texte reçu, nombre compris

Python · exemple autonome

```
import csv
from io import StringIO
source = "quantity,unit_price_tnd\n2,4.500\n"
ligne = next(csv.DictReader(StringIO(source)))
print(ligne["quantity"])
print(type(ligne["quantity"]).__name__)
```

Sortie attendue : 2 puis str. Même si le caractère 2 ressemble à un nombre, DictReader lit ici une chaîne de texte. La mission 03 vérifiera cette chaîne avant sa conversion. Lire et valider sont deux responsabilités distinctes.



Créer docs/exemples/02_csv_texte.py avec ce code. Depuis la racine, lancer sur une ligne : docker compose run --rm atelier python ../docs/exemples/02_csv_texte.py. En route native depuis projet/, remplacer le préfixe par py -3 ou python3.

Vérification manuelle

Compter les lignes de données du fichier initial, sans l'en-tête : 16. Il y aura plus tard 12 faits acceptés. Une lecture qui renvoie seulement 12 lignes a déjà supprimé des observations sans expliquer pourquoi.

MISSION 02 / CONTRAT DE LECTURE.PY

Demander une lecture fidèle du fichier

Ouvrir projet/lecture.py. Demander une fonction qui lit un chemin reçu en argument ; les autres modules pourront ainsi choisir le lot initial ou le lot d'ajout. Une fonction sans chemin qui choisit un fichier caché serait difficile à réutiliser.

Demande 02 · à copier

Écris seulement la fonction lire_csv(chemin) -> liste de dictionnaires de textes. Les imports csv et Path et les chemins sont déjà fournis dans lecture.py ; conserver ROOT fondé sur __file__, catalogues(), ecrire_csv() et main(). Utilise csv.DictReader, séparateur virgule, UTF-8 avec prise en charge du BOM, newline="". Conserve toutes les lignes et colonnes ; ne nettoie, ne convertis et ne filtre rien. Le chemin est fourni par l'appelant. Un fichier absent doit produire une erreur claire. Aucun lancement de traitement à l'import. Le lot initial Atlas a 16 lignes hors en-tête. Explique en trois phrases pourquoi les valeurs restent str.

Lire la réponse

- Repérer with open(...) : le fichier sera fermé après lecture.
- Repérer list(...) : la fonction renvoie les lignes, elle ne renvoie pas seulement un lecteur déjà fermé.
- Refuser une ligne qui saute les erreurs ou filtre les quantités : ce sera le rôle du nettoyage.

Remplacer seulement le bloc de la fonction lire_csv, garder les helpers et main() fournis, enregistrer puis passer au contrôle. Le chemin fondé sur __file__ désigne le dossier du script ; il est plus stable qu'un chemin personnel tapé dans le code.

MISSION 02 / DE L'ENTRÉE À LA SORTIE

Vérifier la première extraction

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python lecture.py
docker compose run --rm atelier python verifier.py 02
```

Le vérificateur appelle votre lire_csv sur les fichiers de projet/data/. Il contrôle une liste de 16 dictionnaires pour le lot initial et les en-têtes attendus. Il ne transforme pas votre code en solution. Vous devez encore montrer la première et la dernière ligne dans l'éditeur.

Symptôme	Vérification utile
15 ou 17 lignes	Avez-vous compté l'en-tête ? Le lecteur a-t-il sauté une ligne ?
Toute la ligne dans une seule clé	Le séparateur ne correspond pas au fichier : il faut une virgule.
Clé sale_id introuvable	Regarder la première clé : BOM ou espace non attendu dans l'en-tête.
FileNotFoundError	Vérifier nom exact, projet/data/ et chemin passé ; ne pas coller un chemin propre à votre poste.

Écrire dans le journal un mini-dictionnaire des données : nom de colonne, sens, exemple, type reçu. quantity : nombre d'articles, exemple « 2 », type reçu str. Ce dictionnaire explique pourquoi les tests de qualité doivent précéder le calcul.

Question de contrôle

Pourquoi « la fonction ne plante pas » est-il insuffisant pour prouver que toutes les lignes ont été lues ? Donner un nombre et une colonne vérifiables.

MISSION 02 / PETITE VARIATION

Changer le fichier, garder le lecteur

Comparer les en-têtes du lot initial et du lot d'ajout dans projet/data/. Les deux décrivent la même sorte de vente ; le second contient 3 lignes de données. Une fonction réutilisable reçoit un autre chemin sans changer son nom ni ses opérations de lecture.

Fichier observé	Question à poser
Lot initial	Combien de lignes reçues ? 16, même si certaines sont incorrectes.
Lot d'ajout	Combien de lignes reçues ? 3, sans décider encore lesquelles sont nouvelles.
Catalogue produits	Quels identifiants sont reconnus ? P01 à P04.
Catalogue boutiques	Quels identifiants sont reconnus ? NBE et TUN.

Sur papier, comparer une ligne ordinaire à une ligne avec des espaces autour d'un code. La lecture doit conserver le texte tel qu'il arrive. Plus tard, normaliser « nbe » en « NBE » permettra de reconnaître la boutique ; supprimer la ligne entière empêcherait de comprendre sa vente.

Pour aller plus loin

Demander à l'IA d'expliquer une seule instruction de lecture que vous ne comprenez pas, avec l'exemple d'un CSV à deux lignes. Ne lui demander aucune nouvelle fonctionnalité tant que le contrat actuel n'est pas vérifié.

Avant de poursuivre, retrouver dans l'arborescence : données originales, fichier Python édité et journal. Ne cherchez pas encore un rapport ; la sortie de cette mission est une extraction fidèle.

MISSION 02 / CONTRÔLER ET REPRENDRE

Jalon 02 · une preuve que je peux expliquer

À ce stade, je sais...

- décrire le grain d'une vente et le rôle des six colonnes ;
- expliquer pourquoi un CSV fournit du texte ;
- faire lire 16 lignes sans supprimer les observations incorrectes.

Conserver une preuve courte

Conserver lecture.py, le contrôle 02 et un dictionnaire des six colonnes dans le journal. Noter aussi les 3 lignes reçues du lot d'ajout.

Expliquer avant de continuer

La source contient une quantité égale à zéro. Faut-il la faire disparaître pendant la lecture ? Expliquer où sa décision et sa raison devront apparaître.

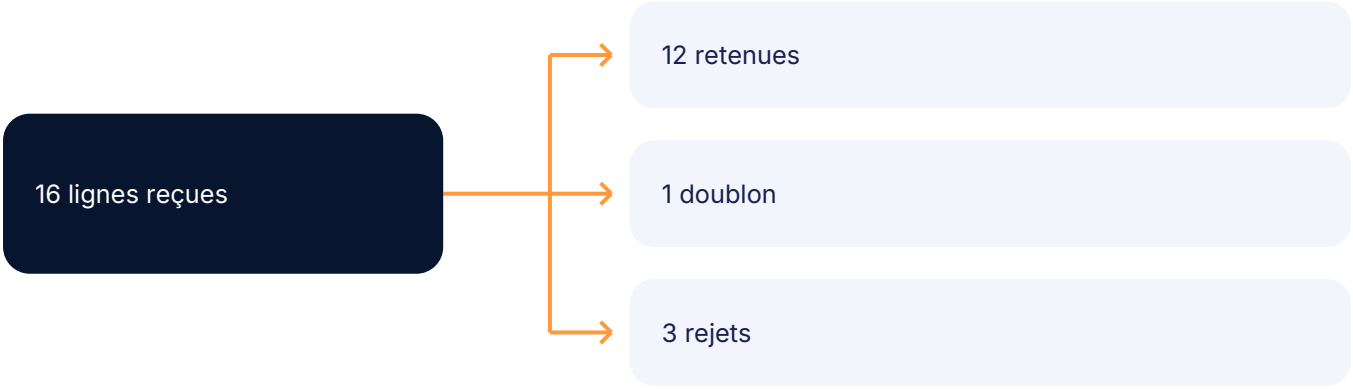
Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 02 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

MISSION 03 / TRANSFORMER SANS DISPARITION

03 · Donner une issue à chaque ligne

La qualité consiste à appliquer des règles annoncées, puis à rendre chaque décision visible. Normaliser répare une forme sans changer le sens, par exemple enlever les espaces autour de nbe puis obtenir NBE. Rejeter explique pourquoi une ligne ne peut pas devenir un fait. Dédupliquer conserve une seule vente quand une répétition identique arrive.



Issue	Signification
Retenue	Valeurs valides et identifiant non encore retenu dans ce fichier.
Doublon	Même sale_id, même contenu normalisé dans le fichier.
Rejet	Valeur invalide ; la ligne et sa raison restent consultables.
Conflit	Même identifiant, contenu différent : signaler explicitement ; ne pas choisir au hasard.

Un bilan avant l'argent

16 = 12 + 1 + 3. Une somme financière correcte avec une ligne perdue n'est pas une preuve suffisante. Vos contrôles portent sur les lignes et sur les montants.

MISSION 03 / TND → MILLIMES

Lire le prix avec son unité exacte

Le texte 4.500 signifie quatre dinars et cinq cents millimes. Pour calculer exactement, Decimal lit le texte, puis nous convertissons en millimes entiers. Le prix est strictement positif et doit représenter un nombre entier de millimes : 4.5000 est équivalent à 4.500 ; 4.5001 est trop précis pour ce contrat.

Python · exemple autonome, prix déjà valide

```
from decimal import Decimal
prix = Decimal("4.500")
prix_millimes = int(prix * 1000)
quantite = 2
montant = quantite * prix_millimes
print(prix_millimes)
print(montant)
```

Sorties : 4500 puis 9000. Cet exemple illustre la conversion d'un prix déjà valide ; il n'est pas encore un validateur. Transformer directement un prix trop précis par int(...) tronquerait une valeur au lieu de la refuser.

Entrée	Décision attendue
quantity = "0" ou "2.5"	Rejet : une quantité doit être un entier positif.
unit_price_tnd = "NaN"	Rejet : le prix doit être fini et positif.
unit_price_tnd = "4.5001"	Rejet : plus précis que le millime.

Créer puis exécuter cet exemple comme le précédent, avec ../docs/exemples/03_millimes.py. Prédire le montant pour 3 articles avant de changer quantite.

MISSION 03 / CONTRAT DE QUALITÉ

Décider avec des règles observables

Champ / situation	Règle
sale_id	Texte non vide ; identifiant stable.
date	Date réelle, exactement au format AAAA-MM-JJ.
shop_id / product_id	Espaces enlevés, majuscules, code reconnu dans le catalogue.
quantity	Entier strictement positif, après lecture du texte.
unit_price_tnd	Decimal fini positif ; valeur exacte en millimes.
Identifiant répété	Contenu normalisé égal : doublon ; différent : conflit explicite.

Dans le catalogue, P01 correspond à Carnet, P02 à Stylo, P03 à Classeur et P04 à Surligneur. NBE correspond à Nabeul et TUN à Tunis. Le catalogue est un dictionnaire de sens, pas un tarif qui remplacerait le prix observé de la vente.

Comparer les contenus après normalisation. Deux dates différentes ou deux quantités différentes pour le même identifiant constituent un conflit. Une modification commerciale réelle demanderait une règle de correction ; elle ne peut pas être inventée par un nettoyage technique.

Avant l'IA

Écrire trois cas en français : un code avec espaces à accepter après normalisation, une quantité nulle à rejeter, une répétition identique à compter une fois. Pour chacun, annoncer son issue et son effet sur le montant.

MISSION 03 / CONTRAT DE NETTOYAGE.PY

Faire produire un nettoyage borné

Demande 03 · à copier

Écris seulement la définition de nettoyer(lignes, produits, boutiques), en Python standard ; conserver les imports, constantes et main() fournis. Cette fonction renvoyant (retenues, rejets, doublons), trois listes. lignes : dictionnaires des six colonnes Atlas ; catalogues : dictionnaires identifiant vers libellé. Enlève les espaces, normalise les codes en majuscules. Vérifie identifiant présent, date ISO réelle exacte, codes connus, quantité entière positive, prix Decimal fini positif représentant un nombre entier de millimes. Les retenues gardent les six champs source normalisés ; quantity est entier, unit_price_tnd est un texte à trois décimales ; ajouter unit_price_millimes et amount_millimes entiers. Les rejets gardent les six champs source et une clé motif non vide ; les doublons ont les huit mêmes champs que les retenues. Garder les répétitions identiques à part. Même sale_id et contenu normalisé différent : lever ValueError explicite. Aucun traitement au moment de l'import. Le bilan initial doit être 12 retenues, 3 rejets, 1 doublon ; somme quantity 32 et somme amount_millimes 99000. Ne code pas ces résultats en dur.

Ouvrir projet/nettoyage.py, lire le retour à trois éléments puis remplacer seulement def nettoyer/TODO et enregistrer la réponse, en gardant le reste du fichier. Le reste du fichier contient déjà les chemins et le lancement : il reste inchangé. Une fonction locale pour valider le prix est possible ; elle ne doit pas remplacer l'interface demandée.

- Suivre une seule ligne de son texte reçu jusqu'à sa sortie.
- Repérer où le motif du rejet est conservé.
- Refuser un except global qui masque toutes les erreurs et renvoie une liste vide.

Si une réponse est longue, demander une explication des trois décisions essentielles : validité, doublon, conflit. Ne remplacez pas tout le projet pour réduire une seule erreur.

MISSION 03 / EXÉCUTER ET CONTREDIRE

Vérifier les comptes et une vente réelle

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python nettoyage.py
docker compose run --rm atelier python verifier.py 03
```

Contrôle	Attendu initial
Lignes reçues	16
Retenues / doublons / rejets	12 / 1 / 3
Quantité totale retenue	32 articles
Montant total retenu	99 000 millimes = 99.000 TND

Choisir une ligne retenue dans le CSV : multiplier sa quantité par son prix en millimes sur papier. Retrouver dans le code l'instruction qui calcule ce même montant. Un total global peut cacher deux erreurs qui se compensent ; ce suivi d'une ligne est une preuve complémentaire.

Sur une copie d'essai de trois lignes, utiliser une ligne valide, sa répétition et une quantité nulle. Prédiction : 1 retenue, 1 doublon, 1 rejet. Revenir ensuite aux sources originales. Une expérience locale qui change les fichiers officiels peut invalider la suite du cours.

Dépannage ciblé

Si le total est 990 au lieu de 99000, vérifier l'unité. S'il y a 13 retenues, vérifier le doublon. Si le nombre de rejets est correct mais sans raison, le journal manque une information essentielle.

MISSION 03 / CONTRÔLER ET REPRENDRE

Jalon 03 · une preuve que je peux expliquer

À ce stade, je sais...

- réconcilier reçu = retenu + doublon + rejet ;
- valider avant de convertir un prix et calculer en millimes ;
- distinguer une répétition identique d'un identifiant conflictuel.

Conserver une preuve courte

Conserver nettoyage.py, la sortie 03 et la prédiction du mini-lot. Dans le journal, détailler un rejet et une normalisation acceptée, avec leur sens métier.

Expliquer avant de continuer

Pourquoi un code avec espaces peut-il être corrigé alors qu'une quantité nulle doit être rejetée ? Quelle règle autorise chacun de ces gestes ?

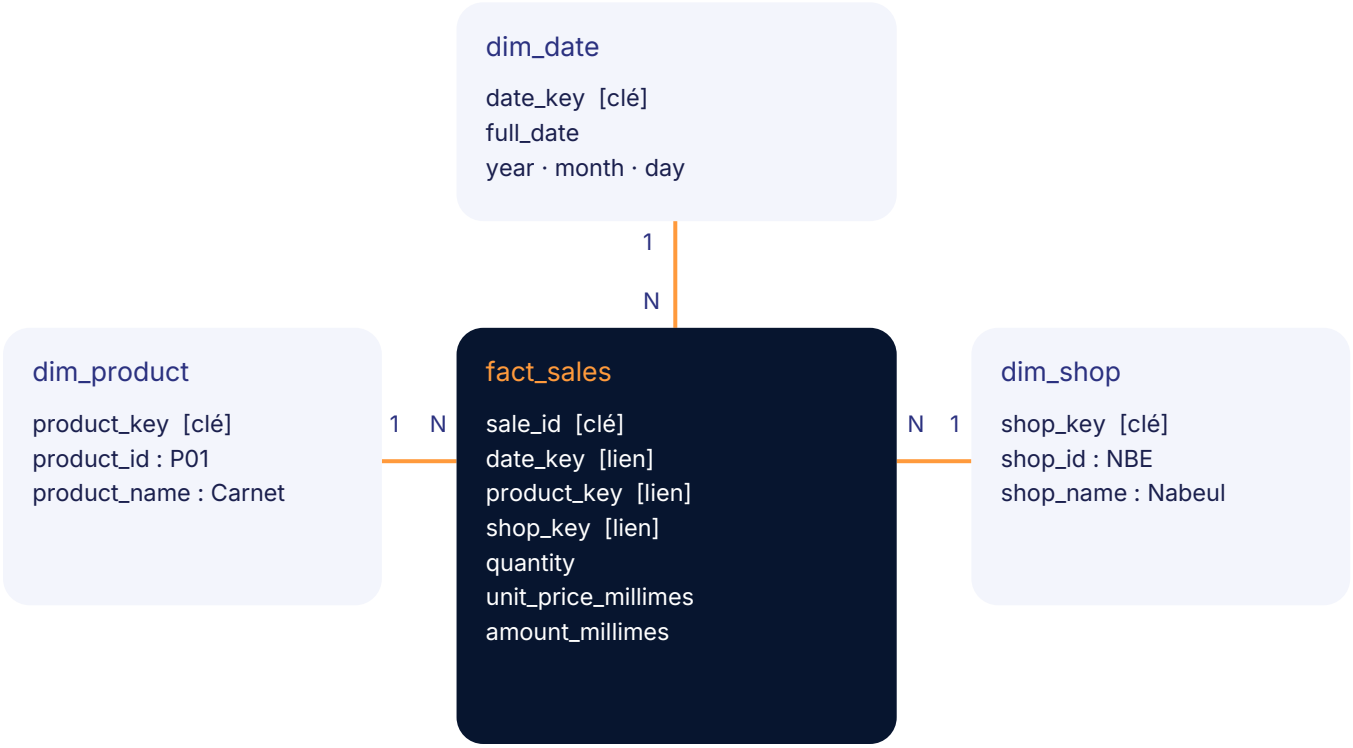
Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 03 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

MISSION 04 / DESSINER AVANT DE CRÉER

04 · Ranger les faits et leur contexte

Une table est un ensemble de lignes ayant les mêmes colonnes. Une clé primaire identifie une ligne sans ambiguïté. Une clé étrangère pointe vers une ligne d'une autre table. Notre fait contient la vente ; les dimensions décrivent son produit, sa boutique et sa date.



1 → N : une ligne de dimension peut décrire plusieurs ventes. Chaque vente retrouve un seul produit, une seule boutique et une seule date.

Le grain reste une vente élémentaire, identifiée par sale_id. Une dimension peut décrire plusieurs ventes : plusieurs faits peuvent concerner le même Carnet. Le schéma est appelé étoile parce que les descriptions entourent la table de faits centrale.

MISSION 04 / NE PAS CONFONDRE LES IDENTIFIANTS

Trois sortes de clés, trois responsabilités

Valeur	Responsabilité
product_id = P01	Clé métier : le code que les CSV utilisent pour désigner Carnet.
product_key = 1	Clé interne : un entier attribué dans la dimension ; sa valeur peut varier selon l'ordre d'insertion.
fact_sales.product_key	Clé étrangère : lien depuis une vente vers le produit de la dimension.
sale_id = S001	Clé du fait : empêche de compter deux fois cette même vente.
date_key = 20260202	Clé de date : AAAAMMJJ, reliée à la date complète et ses attributs.

Ne supposez pas que Carnet aura toujours product_key 1 : le code doit chercher la clé liée à product_id. Le sens métier vient de P01 et de son libellé ; l'entier sert à relier les tables.



Petite expérience sur papier

Dessiner deux ventes de Carnet : elles portent deux sale_id différents mais la même product_key. Ajouter ensuite une vente de Stylo : elle doit pointer vers une autre ligne de dimension. Combien de lignes produit faut-il ? Deux, pas trois.

MISSION 04 / LIRE UN SELECT

Écrire les questions avant SQL

SQL décrit ce que l'on veut lire dans les tables. SELECT choisit des colonnes ou des calculs ; FROM choisit la table ; WHERE garde certaines lignes. COUNT(*) compte les lignes. SUM(quantity) additionne les articles ; SUM(amount_millimes) additionne la valeur des ventes.

SQL · requête de lecture, à utiliser après le chargement

```
SELECT COUNT(*) AS ventes,
       SUM(quantity) AS articles,
       SUM(amount_millimes) AS millimes
FROM fact_sales;
```

Question	Mesure attendue après mission 05
Combien de ventes élémentaires ?	COUNT(*) = 12
Combien d'articles ?	SUM(quantity) = 32
Quel montant ?	SUM(amount_millimes) = 99000

Une table de faits vide contient 0 ligne. SUM sur aucune ligne renvoie NULL dans SQLite ; ce n'est pas un montant. Plus tard, COALESCE(SUM(...), 0) permettra d'afficher zéro. Ne confondez pas ce comportement avec la disparition d'une donnée.

Prédire sans exécuter

Deux ventes portent des quantités 2 et 3. COUNT(*) vaut 2 ; SUM(quantity) vaut 5. Expliquer pourquoi aucune de ces mesures ne donne le nombre de clients.

MISSION 04 / SUIVRE LE LIEN

Une jointure retrouve une description

fact_sales.sale_id	product_key	quantity
A	7	2
B	7	3

dim_product.product_key	product_name
7	Carnet

SQL · illustration à deux tables

```
SELECT p.product_name, SUM(f.quantity) AS articles
FROM fact_sales AS f
JOIN dim_product AS p
  ON p.product_key = f.product_key
GROUP BY p.product_name;
```

La jointure associe chaque fait à son unique description produit. Ici, le résultat est Carnet : 5 articles. GROUP BY réunit les lignes d'un même groupe avant la somme. Une clé produit non unique pourrait multiplier les lignes jointes et gonfler le résultat ; la contrainte UNIQUE protège donc aussi les indicateurs.

Contrôle de compréhension

Si l'on joint sans condition ON, les ventes peuvent être combinées à plusieurs produits. La requête peut s'exécuter sans erreur et produire un montant faux. Comparer le nombre de faits avant et après une jointure est un contrôle utile.

MISSION 04 / CONTRAT DE MODELE.PY

Créer des tables vides, sans effacer

Demande 04 · à copier

Écris seulement la définition de creer_modele(chemin_base), bibliothèque standard. Les imports sqlite3 et Path et connecter() sont déjà fournis et restent intacts. La fonction qui crée le dossier parent si besoin et quatre tables si absentes, sans supprimer une base existante. dim_product(product_key INTEGER PRIMARY KEY, product_id TEXT UNIQUE NOT NULL, product_name TEXT NOT NULL) ; dim_shop(shop_key INTEGER PRIMARY KEY, shop_id TEXT UNIQUE NOT NULL, shop_name TEXT NOT NULL) ; dim_date(date_key INTEGER PRIMARY KEY, full_date TEXT UNIQUE NOT NULL, year INTEGER, month INTEGER, day INTEGER) ; fact_sales(sale_id TEXT PRIMARY KEY, date_key INTEGER NOT NULL REFERENCES dim_date(date_key), product_key INTEGER NOT NULL REFERENCES dim_product(product_key), shop_key INTEGER NOT NULL REFERENCES dim_shop(shop_key), quantity INTEGER CHECK(quantity>0), unit_price_millimes INTEGER CHECK(unit_price_millimes>0), amount_millimes INTEGER CHECK(amount_millimes=quantity*unit_price_millimes)). Active PRAGMA foreign_keys=ON sur la connexion. Fermer proprement. Aucun INSERT et aucun traitement à l'import.

Ouvrir projet/modele.py, remplacer seulement def creer_modele/TODO, garder connecter() et main(), enregistrer puis lancer les commandes. modele.py crée votre base vide ; le vérificateur teste aussi une base temporaire. Le but de ce jalon est la structure, pas encore les 12 ventes.

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python modele.py
docker compose run --rm atelier python verifier.py 04
```

Ne pas réinitialiser par réflexe

CREATE TABLE IF NOT EXISTS permet de relancer sans effacer. Une instruction DROP TABLE ou la suppression de la base n'appartient pas à cette fonction.

MISSION 04 / CONTRÔLER ET REPRENDRE

Jalon 04 · une preuve que je peux expliquer

À ce stade, je sais...

- dessiner la table de faits et les trois dimensions ;
- distinguer clé métier, clé interne et clé étrangère ;
- lire COUNT, SUM, WHERE, JOIN et GROUP BY ;
- créer quatre tables vides sans détruire une base existante.

Conserver une preuve courte

Conserver modele.py, un dessin personnel et la sortie 04. Dans le journal, écrire les résultats prédits d'une jointure de deux ventes vers un produit.

Expliquer avant de continuer

Pourquoi la création d'une table ne charge-t-elle aucune vente ? Quelle table doit exister avant le fait qui la référence ?

Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 04 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

MISSION 05 / CHARGER

05 · Faire entrer les bonnes ventes

Le chargement prend les ventes déjà retenues, crée les descriptions manquantes et insère les faits. Il ne doit ni relire le CSV à l'intérieur de charger, ni refaire des règles incompatibles avec nettoyer. Chaque étape reçoit la sortie de la précédente.

1 Retenues → 2 Dimensions → 3 Clés retrouvées → 4 Faits

Avant insertion du fait	Pourquoi
Produit et boutique présents	Le fait a besoin de leurs clés internes.
Date présente	Le lien date_key doit retrouver une date réelle.
sale_id absent	Il s'agit d'une vente nouvelle.
sale_id déjà présent identique	Compter existing ; conserver un seul fait.
sale_id déjà présent différent	Conflit explicite ; annuler tout le lot.

Notre objectif initial

Après le lot de base : 12 faits, 32 articles, 99 000 millimes. NBE : 43.500 TND ; TUN : 55.500 TND. Les résultats viennent des données, pas d'une constante inscrite dans le chargeur.

MISSION 05 / TOUT ACCEPTER OU TOUT ANNULER

Une transaction protège un lot entier

Une transaction regroupe des écritures : soit tout le lot est validé, soit il est annulé si un conflit survient. Sans cette protection, un programme pourrait charger la première vente puis échouer sur la seconde : la base serait partiellement modifiée.



Essai	Effet attendu
12 ventes nouvelles	inserted = 12 ; existing = 0
Les mêmes 12 à nouveau	inserted = 0 ; existing = 12 ; total inchangé
Une nouvelle puis un conflit	Le conflit est signalé ; aucun fait du lot ne reste ajouté.

Idempotent signifie ici qu'un lot identique rejoué laisse la base dans le même état. Cela ne signifie pas « ignorer toutes les erreurs ». Une valeur changée sous un identifiant déjà connu est une information contradictoire à traiter explicitement.

Prédire un incident

La base contient A. Un lot contient B nouvelle puis A avec une autre quantité. Si le lot échoue, combien de fois B doit-elle être présente ? Zéro. C'est le contrôle du retour arrière.

MISSION 05 / CONTRAT DE CHARGEMENT.PY

Faire produire le chargement

Demande 05 · à copier

Écris seulement la définition de charger(ventes, produits, boutiques, chemin_base) ; conserver les imports, connecter(), total_base(), lire_ventes_nettoyees() et main() fournis. La fonction renvoyant {"inserted": nombre, "existing": nombre}. ventes : retenues normalisées de nettoyer ; produits et boutiques : dictionnaires identifiant vers nom. La base a déjà été créée à la mission 04 ; si elle manque, lever FileNotFoundError. Utiliser connecter() fourni. Créer puis retrouver les dimensions par leurs codes ; date_key est AAAAMMJJ. Insérer les faits avec quantity, unit_price_millimes et amount_millimes. Active foreign_keys sur ta propre connexion. Un sale_id absent est inséré ; s'il existe avec date, produit, boutique, quantité et prix égaux, compter existing. S'il existe avec un contenu différent, lever ValueError et annuler toutes les écritures de ce lot. Une seule transaction de chargement ; pas de suppression, pas de INSERT OR REPLACE qui masque un conflit. Fermer proprement, aucun traitement à l'import. Base initiale : 12 faits, 32 articles, 99000 millimes ; second chargement identique : 0 insertion.

Ouvrir projet/chargement.py, lire les branches nouveau / identique / conflictuel. Enregistrer le code puis lancer le vérificateur 05. Les données retenues arrivent en argument ; l'IA n'a pas besoin d'un service cloud ou d'une bibliothèque ETL externe.

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python chargement.py
docker compose run --rm atelier python verifier.py 05
```

Si une réponse invente un module absent, lui demander de reprendre uniquement charger avec sqlite3 et les interfaces déjà construites. Ne pas installer une bibliothèque pour contourner une dépendance qu'on n'a pas demandée.

MISSION 05 / TROIS PREUVES COMPLÉMENTAIRES

Suivre un fait du CSV à SQLite

Choisir S001 dans le fichier source. Dans votre journal, noter date, boutique, produit, quantité, prix et montant. Expliquer ensuite le chemin : la lecture fournit du texte ; nettoyer valide et calcule ; charger retrouve trois clés et insère un fait.

Contrôle	Attendu
COUNT(fact_sales)	12 ventes élémentaires
SUM(quantity)	32 articles
SUM(amount_millimes)	99 000 millimes
PRAGMA foreign_key_check	[] : aucune référence orpheline
Rejeu du lot initial	0 nouveau fait ; 12 déjà présents

PRAGMA foreign_key_check examine les liens de la base. Une liste vide est un bon résultat, mais ne prouve pas le bon calcul des montants. Les comptes, les sommes et l'intégrité des liens répondent à des questions différentes.

Si le nombre est bon mais la somme fausse

Vérifier la quantité et le prix de la vente suivie. Comparer le montant avant et après chargement. Ne recalculer pas le prix à partir du catalogue : le catalogue décrit les produits, il ne remplace pas le prix vendu.

Les modules exécutés produisent vos sorties ; le vérificateur les lit et teste aussi vos fonctions sur de petits cas temporaires. L'orchestration fournie et le rapport seront étudiés à la mission 07.

MISSION 05 / REGROUPER SANS DÉFORMER

Poser une question utile à la base

SQL · à lire, utilisé ensuite dans le rapport

```
SELECT s.shop_id,
       SUM(f.amount_millimes) AS millimes
FROM fact_sales AS f
JOIN dim_shop AS s ON s.shop_key = f.shop_key
GROUP BY s.shop_id
ORDER BY s.shop_id;
```

Boutique	Montant initial
NBE	43 500 millimes = 43.500 TND
TUN	55 500 millimes = 55.500 TND
Réconciliation	43 500 + 55 500 = 99 000 millimes

Avant d'exécuter une requête, écrire sa question en français. La somme par boutique doit se réconcilier avec le total global si chaque vente appartient à une et une seule boutique. Une différence révèle un filtre, un lien ou un périmètre à examiner.

Prédire une variation sur papier : ajouter 2 stylos à 1.500 TND dans NBE augmenterait son montant de 3.000 TND et le global du même montant. Ce raisonnement prépare le lot d'ajout ; ne modifier pas les données officielles pour obtenir ce cas.

Si je termine tôt

Écrire la même question par produit et vérifier les quatre montants : Carnet 22.500, Stylo 18.000, Classeur 36.000, Surligneur 22.500 TND. Leur somme doit être 99.000 TND.

MISSION 05 / CONTRÔLER ET REPRENDRE

Jalon 05 · une preuve que je peux expliquer

À ce stade, je sais...

- expliquer pourquoi les dimensions sont chargées avant les faits ;
- distinguer transaction, insertion et rejeu ;
- contrôler 12 faits, 32 articles, 99.000 TND et des clés étrangères sans anomalie.

Conserver une preuve courte

Conserver chargement.py, la sortie 05, le parcours commenté de S001 et une phrase sur le résultat vide de foreign_key_check.

Expliquer avant de continuer

Un lot ajoute une vente puis rencontre un identifiant conflictuel. Pourquoi un simple message d'erreur ne suffit-il pas si la première vente est restée enregistrée ?

Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 05 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

MISSION 06 / L'INCRÉMENTAL

06 · Le prochain fichier arrive

Un ETL utile doit pouvoir recevoir plusieurs lots. Le lot d'ajout Atlas contient 3 lignes : 2 ventes nouvelles et 1 déjà chargée. Ces trois lignes peuvent être valides dans le fichier, tout en n'ajoutant que deux faits à la base.

Niveau de contrôle	Question
Dans le nouveau CSV	Combien de lignes reçues, retenues, rejetées ou répétées dans ce fichier ?
Face à la base	Parmi les retenues, lesquelles sont nouvelles et lesquelles existent déjà ?
Après chargement	Combien de faits, d'articles et de millimes possède l'entrepôt ?



Une vente déjà en base n'est pas forcément un doublon à l'intérieur du fichier reçu. Garder ces deux comptes distincts permet de comprendre un lot correct qui ne change presque pas le total.

Prédiction avant l'IA

12 faits + 2 nouvelles ventes = 14 faits. Le résultat attendu est 35 articles et 115.500 TND. Lors du second passage de ce lot, les trois lignes sont déjà connues : aucune insertion.

MISSION 06 / CONTRAT DE INCREMENTAL.PY

Réutiliser le pipeline plutôt que le dupliquer

Demande 06 · à copier

Écris seulement la fonction ajouter(chemin_source, chemin_base, chemin_data), sans remplacer les imports ni main() fournis. Utiliser catalogues(chemin_data) et ecrire_csv() de lecture déjà importés. Utilise lire_csv de lecture, nettoyer de nettoyage et charger de chargement. Lis les catalogues du dossier chemin_data et transforme-les en dictionnaires identifiant vers nom. Le chemin_source désigne le lot à traiter ; aucun chemin personnel en dur. Nettoyer le lot, charger les retenues et renvoyer exactement un dictionnaire de six clés : raw, accepted, rejected, duplicates, inserted, existing, toutes entières. Écrire rejets_ajout.csv et doublons_ajout.csv à côté de la base avec ecrire_csv(), CHAMPS_SOURCE + ["motif"] et CHAMPS_VENTES déjà fournis. Réutiliser les règles existantes, ne pas créer une seconde version du validateur. Propager un conflit avec un message utile ; ne pas supprimer la base. Aucun traitement à l'import. Lot d'ajout : 3 reçues, 3 retenues, 0 rejet, 0 doublon interne, puis 2 inserted et 1 existing ; son rejeu : 0 inserted, 3 existing.

Ouvrir projet/incremental.py. Joindre seulement les en-têtes des catalogues si l'IA demande leur structure, et lire les noms de fichiers dans projet/data/. La sortie doit séparer les comptes de nettoyage de ceux de chargement. Enregistrer puis lancer :

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python incremental.py
docker compose run --rm atelier python verifier.py 06
```

Revoir la responsabilité

ajouter orchestre des fonctions déjà contrôlées. Si elle reprogramme toute la qualité dans un second bloc, demander de réutiliser nettoyer et charger pour éviter des règles divergentes.

MISSION 06 / AVANT → APRÈS → APRÈS

Prouver l'ajout et son rejeu

État	Faits	Articles	Montant TND
Après base	12	32	99.000
Après ajout	14	35	115.500
Après rejeu du même ajout	14	35	115.500

Le gain est de 3 articles et 16.500 TND. Le rejeu ne doit rien augmenter. Garder trois lignes d'état dans le journal avec les résultats réels : elles montrent la différence entre un ajout utile et une répétition inoffensive.

Le module incremental.py modifie votre base et effectue son rejeu ; le vérificateur teste aussi votre fonction dans une base temporaire et lit ensuite vos sorties existantes. Son rôle est de détecter l'erreur de comportement. Lorsque le contrôle est vert, lire encore les nombres : un message vert sans compréhension de 2 nouvelles / 1 existante ne suffit pas à expliquer le système.

Pourquoi la date ne remplace pas l'identifiant

Deux ventes peuvent partager la même date, le même produit et la même quantité. Les fusionner sur ces seules valeurs supprimerait une vente réelle. sale_id donne l'identité ; les autres champs permettent de contrôler son contenu.

Une demande IA utile contient l'interface, l'erreur, attendu et obtenu. Exemple : « Au rejeu j'ai inserted=3 et 17 faits, attendu inserted=0 et 14. Voici la comparaison d'identité dans charger. Corrige cette comparaison seulement. »

MISSION 06 / LE CONFLIT CONTRÔLÉ

Tester une contradiction sans casser sa base

Un identifiant stable ne doit pas changer silencieusement de sens. Pour l'observer, partir d'une copie de travail complète destinée aux essais, puis d'une copie du lot. Modifier uniquement la quantité d'une vente déjà connue en gardant son sale_id. Noter la valeur initiale et la valeur proposée.



- Prédire le refus avant de lancer le test.
- Conserver le message de conflit et vérifier que le nombre de faits et la somme ne changent pas.
- Si le lot contient d'abord une nouvelle vente, vérifier qu'elle n'est pas restée insérée après le conflit.
- Revenir au dossier principal sans remplacer ses données originales.

Une bonne question à l'assistant

« Le même sale_id apparaît avec une quantité différente. Je veux un refus explicite et un retour arrière du lot entier. Montre la comparaison manquante dans charger et propose une correction de cette fonction seulement. »

Cette règle protège notre exercice contre une correction involontaire. Un véritable système commercial pourrait gérer des rectifications par versions ou annulations, mais il lui faudrait un contrat métier explicite. Notre module n'invente pas cette politique.

MISSION 06 / DIAGNOSTIQUER AVANT DE DEMANDER

Lire une erreur pour choisir une action

Un traceback est la trace des appels menant à l'erreur. Lire d'abord la dernière ligne : elle nomme la catégorie et le message. Puis remonter au dernier fichier de votre projet. Une erreur dans le SQL demande un autre contrôle qu'un fichier source absent.

Message	Prochaine observation
FileNotFoundError	Nom du lot et dossier de départ ; le chemin existe-t-il ?
KeyError	Comparer clé demandée et en-tête reçu, sans renommer au hasard.
IntegrityError	Vérifier clé étrangère, unicité ou contrainte de montant.
ValueError de conflit	Comparer identifiant et contenu aux données déjà chargées.
NotImplementedError	Le fichier du jalon reste à construire.

Une correction efficace modifie une cause puis relance le même contrôle. Après une correction de charger, relancer 05 puis 06 : l'ajout dépend du comportement initial. Ne changez pas plusieurs modules sans pouvoir attribuer l'effet à une modification.

Quand l'IA n'est plus disponible

Réduire le cas à deux lignes, lire les instructions concernées, consulter l'indice 06, travailler sur le journal et la prédiction. Vous pouvez continuer l'apprentissage sans nouvelle génération. La reprise complète est une aide séparée et garde votre tentative intacte.

MISSION 06 / CONTRÔLER ET REPRENDRE

Jalon 06 · une preuve que je peux expliquer

À ce stade, je sais...

- séparer les doublons du fichier des ventes déjà en base ;
- réutiliser les fonctions contrôlées pour un nouveau lot ;
- prouver 14 faits, 35 articles, 115.500 TND et un rejeu stable ;
- refuser un conflit sans laisser un lot partiellement chargé.

Conserver une preuve courte

Conserver incremental.py, la sortie 06, le tableau avant/après/rejeu et une trace réelle du refus conflictuel sur une copie d'essai.

Expliquer avant de continuer

Pourquoi « ne charger que les dates nouvelles » supprimerait-il potentiellement une vente ? Donner un exemple où l'identifiant est nécessaire.

Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 06 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

MISSION 07 / PRÉSENTER UNE OBSERVATION

07 · Un rapport dont je peux retrouver la source

Un rapport n'est pas seulement une page jolie. Il indique son périmètre, ses unités, sa période et l'origine de ses nombres. Notre HTML sera produit depuis SQLite après base et ajout. Il doit donc refléter 14 ventes élémentaires, 35 articles et 115.500 TND.

1 SQLite → 2 Requêtes SQL → 3 Valeurs contrôlées → 4 HTML local

À montrer	Pourquoi
Période réellement observée	Éviter de présenter une courte période comme une année.
Nombre de faits et articles	Distinguer activité élémentaire et quantité vendue.
Montant TND à trois décimales	Préserver les millimes et préciser l'unité.
Par boutique et par produit	Répondre à une question avec des libellés lisibles.
Source et limites	Expliquer ce que le lecteur peut raisonnablement conclure.

Prédire un titre honnête

« Ventes observées par boutique » convient. « Boutique la plus rentable » ne convient pas : aucun coût n'est disponible. L'IA peut proposer un titre séduisant qui dépasse les données.

MISSION 07 / CONTRAT DE RAPPORT.PY

Faire construire le rapport depuis la base

Demande 07a · à copier

Écris seulement la définition de generer(chemin_base, chemin_html) ; conserver les imports, SQL_BOUTIQUES, SQL_PRODUITS, connecter(), formater_tnd(), lignes_table() et main() fournis. La fonction qui interroge les quatre tables Atlas avec sqlite3, écrit un HTML autonome UTF-8 et renvoie exactement les clés facts, quantity, amount_millimes, period_start, period_end, shops, products. Les deux dernières sont des listes de dictionnaires id/name/quantity/amount_millimes ; les périodes sont des textes ISO. facts, quantity et amount_millimes sont entiers. Créer le dossier de sortie si besoin. Calculer depuis la base le nombre de faits, la quantité, le montant en millimes, les dates min/max et les montants par boutique et produit avec leurs libellés. Utiliser SUM, COUNT et JOIN avec les clés ; ne pas inscrire 115500 en dur. Afficher les TND avec trois décimales, tableaux lisibles, titre, période, unité, source et limites : ventes observées, sans stock/coût/marge/prévision. Échapper les libellés HTML, sans CDN, image distante, JavaScript externe ou accès réseau. Aucun traitement à l'import.

Ouvrir projet/rapport.py, remplacer uniquement le bloc def generer/TODO puis lire une requête et son affichage. Les calculs gardent des millimes entiers ; seule la présentation les transforme en TND. Le rapport peut avoir une mise en page différente de l'exemple tant que le sens et les contrôles sont respectés.

Pourquoi sans service externe

Le rapport doit rester consultable sur le poste et transportable dans le dossier de travail. Une police ou un graphique téléchargé à l'ouverture ajouterait une dépendance inutile à cette preuve.

MISSION 07 / CONTRAT DE EXECUTER.PY

Assembler le bouton de relance

Les sept fonctions construites sont maintenant utilisables ensemble. Ouvrir projet/executer.py : ce fichier d'assemblage est fourni. Il évite de redemander à l'IA un nouveau pipeline complet qui risquerait de changer vos interfaces.

- Lire executer() de haut en bas et retrouver les appels de lecture, nettoyage, modèle, chargement, ajout et rapport.
- Repérer le lot initial puis les deux appels à ajouter : le second est un rejeu contrôlé.
- Repérer la base outputs/atlas.sqlite et le rapport outputs/rapport_atlas.html. Les chemins partent du dossier du programme.
- Repérer if __name__ == "__main__" : importer le fichier ne doit pas lancer le programme.

Demande 07b · expliquer un assemblage

Voici les appels de executer() déjà fourni. Explique en cinq phrases maximum leur ordre et la raison du deuxième appel à ajouter. Ne modifie aucun fichier et ne propose aucun nouveau helper. Je contrôlerai moi-même 14 faits, 35 articles et 115500 millimes après relance.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python executer.py
docker compose run --rm atelier python verifier.py 07
```

Conserver les bilans réellement imprimés. Au premier lancement sur une base déjà construite par les missions, les ventes du lot initial existent déjà : inserted peut donc être 0. Ce qui doit rester stable est l'état final, pas la répétition de 12 nouvelles insertions.

MISSION 07 / LE RAPPORT DANS LE NAVIGATEUR

Ouvrir et vérifier ce qui est vraiment livré

Terminal · racine du cours

```
docker compose up -d
```

Ouvrir ensuite http://localhost:8000/rapport_atlas.html dans le navigateur. Le service web sert le dossier outputs de projet/. En mode natif, ouvrir directement projet/outputs/rapport_atlas.html par double-clic, ou lancer depuis projet/ : py -3 -m http.server 8000 --directory outputs ; sous Ubuntu remplacer py -3 par python3.

- Comparer le total du HTML au récapitulatif réellement imprimé par executer.py.
- Vérifier la période dans les données, le nombre de faits, les articles et les unités.
- Retrouver deux montants par boutique ou produit dans les résultats SQL.
- Fermer ou arrêter le serveur après la séance : docker compose down ; en mode natif Ctrl+C dans le terminal du serveur.

La page montre un ancien résultat

Relancer executer.py, puis actualiser le navigateur. Vérifier le chemin indiqué dans le terminal ; deux copies du cours peuvent contenir deux rapports différents. Ne pas corriger les chiffres directement dans le HTML.

Lancer executer.py une seconde fois. Le total final doit rester 14 / 35 / 115.500. Un succès d'affichage et un succès de rejeu sont deux preuves différentes.

MISSION 07 / DU CHIFFRE À L'EXPLICATION

Écrire une conclusion et préparer la reprise

Votre conclusion comporte trois phrases : une observation avec valeur et unité ; le chemin qui permet de la vérifier ; une limite. Pour la base initiale, TUN représente 55.500 TND contre 43.500 pour NBE. Ces valeurs décrivent des ventes, sans expliquer les coûts ni la demande future. Pour le rapport final, utiliser ses valeurs calculées après ajout.

Créer projet/ETAT_PROJET.md

- Objectif et grain : une vente élémentaire par sale_id.
- Dernier contrôle réussi et commande exacte de relance.
- Fichiers modifiés et résultat attendu : 14 faits, 35 articles, 115.500 TND.
- Prochain blocage reproduit, ou prochaine amélioration si le socle est terminé.
- Aucune clé, aucun mot de passe et aucune donnée personnelle dans cet état.

Créer aussi projet/README.md : dossier de départ, commande Docker, variante native, emplacement des sources et sorties, commande de rejeu. Faire lire ces consignes à un pair. S'il ne sait pas où ouvrir le terminal, améliorer la consigne plutôt que faire le geste à sa place.

Passer à une nouvelle conversation IA

Fournir ETAT_PROJET.md, l'interface de la fonction concernée et le plus petit exemple reproduisant le problème. Le nouveau contexte n'a pas besoin de toute l'histoire du cours.

MISSION 07 / CONTRÔLER ET REPRENDRE

Jalon 07 · une preuve que je peux expliquer

À ce stade, je sais...

- produire un rapport autonome à partir de SQLite ;
- contrôler période, unité et chiffres avant de formuler une conclusion ;
- relancer le projet et transmettre un état de reprise utile.

Conserver une preuve courte

Conserver rapport.py, executer.py, les journaux et le rapport issus de l'exécution. Ajouter une sortie du second lancement, README.md et ETAT_PROJET.md.

Expliquer avant de continuer

Montrer un chiffre du rapport, sa requête puis une vente source qui y contribue. Dire une conclusion que ce même chiffre ne permet pas de justifier.

Dans projet/journal_apprentissage.md, noter : prédiction, commande exacte, résultat observé, cause de l'écart éventuel et prochaine action. Une sortie rouge est utile si vous savez la reproduire et la réduire.

Si le jalon bloque, ouvrir seulement l'indice de mission 07 dans 05_indices_et_corriges.pdf. Si la limite de l'assistant est atteinte, poursuivre les calculs, la lecture et les tests. La fiche de reprise en fin de cours indique comment obtenir une copie de secours sans écraser votre travail.

REPRENDRE / SANS ÉCRASER MES FICHIERS

Je suis bloqué : une reprise qui garde ma tentative

Commencer par écrire le blocage : mission, fichier, commande, résultat attendu et obtenu. Lire l'indice correspondant dans le livre 05. Si vous devez repartir d'un jalon fonctionnel, l'outil fourni copie une référence dans un nouveau dossier : votre projet reste intact.

Natif Windows · racine du cours

```
py -3 outils/jalon.py 03 --destination reprise-03
```

Sous Ubuntu natif : remplacer py -3 par python3. Avec Docker, depuis la racine saisir sur une ligne : docker compose run --rm -w /workspace atelier python outils/jalon.py 03 --destination reprises/reprise-03. Le livre 05 explique ensuite comment exécuter cette copie dans le même conteneur.

Lire le chemin du nouveau dossier annoncé par l'outil et y ouvrir le projet. Choisir le jalon qui précède votre blocage, pas le projet final par défaut. Le livre 05 explique le mécanisme et les alternatives natives. Si le dossier de destination existe déjà, l'outil doit refuser de l'écraser.

- Conserver l'ancienne tentative et son journal pour expliquer l'erreur.
- Comparer un fichier à la fois entre la tentative et la référence.
- Rejouer le contrôle du jalon dans le nouveau dossier avant de poursuivre.
- Noter les éléments repris et ce que vous avez compris du correctif.

Reprendre n'est pas prétendre avoir construit

Une reprise permet d'apprendre la suite. Votre preuve doit distinguer ce que vous avez écrit, ce que vous avez repris et la variation que vous savez maintenant expliquer.

RECAP / DU CONTRAT À LA LIVRAISON

Carte du projet que j'ai construit

Fichier	Entrée → sortie / responsabilité
calcul.py	Ventes chiffrées → total entier en millimes.
lecture.py	Chemin CSV → liste de dictionnaires de textes.
nettoyage.py	Lignes + catalogues → retenues, rejets, doublons.
modele.py	Chemin de base → quatre tables si absentes.
chargement.py	Retenues + catalogues → faits et compte du chargement.
incremental.py	Nouveau lot → qualité puis chargement réutilisés.
rapport.py	Base → HTML et mesures calculées.
executer.py	Orchestration locale avec chemins stables et bilans.
verifier.py	Contrôles publics de vos fonctions ; à lire, sans modifier.

Le modèle en étoile rend le contexte d'une vente accessible sans répéter ses descriptions partout. L'idempotence permet de répéter une exécution sans multiplier l'activité. La traçabilité conserve les raisons d'exclusion. Ce sont des propriétés que vous pouvez prouver, indépendamment du style du code proposé par l'IA.

Terminal · racine du dossier du cours

```
docker compose run --rm atelier python verifier.py tout
```

Exécuter ce contrôle après toute modification transversale. Un résultat vert atteste les comportements vérifiés ; il ne prouve pas que toutes les futures données possibles ont été couvertes.

AUTO-CONTRÔLE / SANS NOUVELLE RÉPONSE IA

Avant le défi : vérifier mon autonomie

Je dois pouvoir...	Preuve courte
Prédire un montant	Recalculer quantité × prix en millimes.
Expliquer une exclusion	Montrer la ligne, la règle et la raison conservée.
Lire le modèle	Suivre un fait vers produit, boutique et date.
Interpréter un SQL	Distinguer COUNT des ventes et SUM des articles.
Prouver le rejeu	Comparer faits, quantité et montant avant/après.
Réduire un défaut	Isoler un cas puis vérifier une correction.
Défendre un rapport	Citer source, période, unité et limite.

Choisir deux lignes de votre journal : une réussite et une erreur réparée. En deux minutes, expliquer ce qui était prévu, ce qui s'est produit et ce qui a permis de décider. Un pair peut poser une question ; vous pouvez consulter votre code et vos notes.

Prochaine étape

Ouvrir 04_defi_campus_box.pdf. Le cas change : des portions de repas, un statut et un canal apparaissent. Vous réutiliserez la méthode de construction, mais vous devrez adapter le contrat avant de réutiliser le code.

TRANSFÉRER / 7 HEURES EN ÉQUIPE DE 1 À 3

CampusBox : préparer le travail collectif

Le défi complet est dans le livre 04 et defi/student_challenge.md. CampusBox veut mesurer les ventes de repas servis, par campus et par canal, sans double comptage. Les lignes ANNULEE et EN_ATTENTE restent hors indicateurs et sont expliquées dans le journal. Le canal COMPTOIR ou EN_LIGNE devient une information à conserver.

1 Contrat → 2 Chemin initial → 3 Ajout + rejeu → 4 Démonstration

Jalon	Budget	Preuve
Définir	60 min	Grain, règles, petit calcul et modèle.
Construire	150 min	Base initiale, journal et contrôles.
Éprouver	120 min	Ajout, rejeu stable et conflit refusé.
Expliquer	90 min	Rapport, consigne de relance et preuve individuelle.

Les rôles demandeur, opérateur et testeur tournent à chaque jalon. À deux, une personne cumule deux rôles avant inversion ; seul, vous notez les trois points de vue. Chaque membre doit savoir expliquer une règle et une preuve, même si un seul clavier a été utilisé.

À ce stade, je sais...

repartir d'un contrat différent, choisir ce qui se réutilise et annoncer ce qui doit être modifié avant de demander du code.

ACQUIS / CONTINUER AVEC UNE PETITE VARIATION

Après ces 28 heures

Vous avez construit un système local qui reçoit des fichiers, contrôle leur qualité, conserve des faits reliés à leur contexte et produit une information vérifiable. L'IA a pu accélérer l'écriture, mais votre contribution décisive est le contrat, la prédiction, l'exécution et l'explication.

- Pour continuer : ajouter une question SQL sur une copie, avec un résultat prédit avant de générer le code.
- Pour un autre jeu de données : définir d'abord le grain, les unités, l'identité et le périmètre des indicateurs.
- Pour une automatisation réelle : préciser fréquence, droits, sauvegardes et conduite à tenir en cas de données contradictoires.

Une prochaine amélioration justifiée

Choisir un besoin précis, par exemple comparer les quantités par produit et par jour. Écrire les données nécessaires, un petit calcul de contrôle et une limite. Une amélioration utile commence par une question, pas par une dépendance technique.

Avant de fermer : enregistrer le journal et ETAT_PROJET.md, garder les sources intactes, arrêter le serveur local et vérifier que votre dossier peut être retrouvé. Votre preuve finale comprend les fichiers qui ont réellement produit le rapport, pas seulement sa capture.